Advanced Artificial Intelligence and Machine Learning Lab 9

November 8, 2026

1 Lab 9

> Adapted from the Hugging Face Deep Reinforcement Learning Course (`huggingface/deep-rl-class`, commit `dbc53d8`, Apache 2.0), notebooks Unit 2 ("Q-Learning with FrozenLake-v1 and Taxi-v3") and Unit 4 ("Policy Gradient with PyTorch"). Code and prose are modified and shortened. Changed from the source: no Hugging Face Hub login, upload or replay videos (so no `pygame`, `pyvirtualdisplay` or `imageio`); Unit 4 ported from `gym` to `gymnasium`; REINFORCE trained for 500 instead of 1,000 episodes; Exercise 4 (several seeds) is new.

Session 11 introduced two families of RL methods. **Value-based** methods learn an action-value function $Q(s, a)$ and act greedily with respect to it - Q-learning is the classic example. **Policy-based** methods parametrise the policy $\pi_\theta(a|s)$ directly and follow the policy gradient - REINFORCE is the classic example. In this lab you implement both from scratch: tabular Q-learning on two small grid worlds, then REINFORCE with a small neural network on CartPole.

> **Instructor timing (2h session).** ~10 min: intro and setup. ~35 min: Exercise 1 - Q-table, greedy and ϵ-greedy policies, the Q-learning update, on FrozenLake. ~15 min: Exercise 2 - the same agent on Taxi, plus the slippery FrozenLake. ~40 min: Exercise 3 - the REINFORCE policy network, the returns and the loss, and one training run (1-2 min). ~15 min: Exercise 4 - two more seeds and a look at the spread (3-5 min of training; discuss the questions while it runs). ~5 min buffer.

1.1 Setup

These labs are designed to run on Google Colab - no local Python install needed. Open this notebook via Colab's GitHub loader (`colab.research.google.com/github/KarikS/lecture_advanced_ai_ml/blob/main/<path-to-this-notebook`) or File > Open notebook > GitHub tab, repo `KarikS/lecture_advanced_ai_ml`, then run the cell below once per session. Running the notebook locally instead (e.g. via the `exercises/python/` venv described in that folder's README) works unchanged - the cell below is a no-op there. No GPU is needed: the Q-learning exercises run in seconds, and one REINFORCE run takes 1-2.5 minutes on a single CPU core.

[1]:
```
import sys

IN_COLAB = 'google.colab' in sys.modules

if IN_COLAB:
```

```
    import os
    if not os.path.exists('lecture_advanced_ai_ml'):
        !git clone https://github.com/KarikS/lecture_advanced_ai_ml.git
    !pip install -q gymnasium
```

1.2 Imports

[2]:
```
import random
import time
from collections import deque

import gymnasium as gym
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn.functional as F
from matplotlib_inline.backend_inline import set_matplotlib_formats
from torch import nn
from torch.distributions import Categorical

set_matplotlib_formats('png', 'pdf')
random.seed(0)
np.random.seed(0)
torch.manual_seed(0)
# The policy network below is tiny; one thread is faster than several for it.
torch.set_num_threads(1)
```

1.3 Exercise 1: Q-Learning on FrozenLake

FrozenLake is a 4×4 grid. The agent starts at `S` and has to reach the goal `G`, walking on frozen tiles `F` and avoiding the holes `H`. We start with the non-slippery version, in which every action moves the agent in the intended direction (a deterministic MDP). `render_mode="ansi"` lets us print the grid as text.

[3]:
```
env = gym.make('FrozenLake-v1', map_name='4x4', is_slippery=False,␣
 ↪render_mode='ansi')
env.reset(seed=0)
env.action_space.seed(0)
print(env.render())
print('Observation space:', env.observation_space)
print('Action space:', env.action_space)
```

```
SFFF
FHFH
FFFH
HFFG
```

```
Observation space: Discrete(16)
Action space: Discrete(4)
```

A state is an integer: the agent's position `row * 4 + col`, so the 16 states are 0, ..., 15 and the goal is $3 \cdot 4 + 3 = 15$. The 4 actions are 0: left, 1: down, 2: right, 3: up. The reward is +1 for reaching the goal and 0 otherwise.

In the vocabulary of Session 11: we do not know the model $\mathcal{P}^a_{ss'}$, $\mathcal{R}^a_s$ of this MDP, so we cannot solve the Bellman optimality equation directly. Q-learning learns $q_*(s, a)$ from experience instead. With only 16 states and 4 actions, Q fits in a lookup table - the **Q-table**, one row per state, one column per action.

[4]:
```
def initialize_q_table(state_space: int, action_space: int) -> np.ndarray:
# TODO: Return a (state_space, action_space) array of zeros.
```

[5]:
```
state_space = env.observation_space.n
action_space = env.action_space.n
Qtable_frozenlake = initialize_q_table(state_space, action_space)
print(Qtable_frozenlake.shape)
assert Qtable_frozenlake.shape == (16, 4)
```

Q-learning is **off-policy** (Session 11, "Off-Policy Control with Q-Learning"): it uses two policies.

- The **target policy** is greedy with respect to Q: it takes the action with the highest Q-value. It is also the policy we end up with after training.
- The **behaviour policy**, which actually collects the experience, is ϵ-greedy: with probability $1 - \epsilon$ it takes the greedy action, with probability ϵ a random one. This keeps the agent exploring.

[6]:
```
def greedy_policy(Qtable: np.ndarray, state: int) -> int:
# TODO: Return the action with the highest Q-value in this state (np.argmax).


def epsilon_greedy_policy(Qtable: np.ndarray, state: int, epsilon: float, env:␣
↪gym.Env) -> int:
# TODO: With probability 1 - epsilon return the greedy action, otherwise a random
# action. Hint: random.uniform(0, 1) and env.action_space.sample().
```

Now the training loop. For every episode we decay ϵ (more exploration at the start, more exploitation later), reset the environment, and then repeat until the episode ends:

1. choose an action A_t with the ϵ-greedy policy,
2. take it and observe R_{t+1} and S_{t+1},
3. update $Q(S_t, A_t)$ towards the Q-learning target:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right)$$

`env.step` returns five values: the new state, the reward, `terminated` (the episode ended inside the MDP: goal or hole), `truncated` (it was cut off by a time limit) and an info dict.

[7]:
```
def train(n_training_episodes: int, min_epsilon: float, max_epsilon: float,␣
↪decay_rate: float,
          env: gym.Env, max_steps: int, Qtable: np.ndarray, learning_rate: float,
          gamma: float) -> np.ndarray:
    for episode in range(n_training_episodes):
        epsilon = min_epsilon + (max_epsilon - min_epsilon) * np.exp(-decay_rate␣
↪* episode)
        state, info = env.reset()

        for step in range(max_steps):
# TODO: Choose an action with the epsilon-greedy policy, take it with
# env.step, and update Qtable[state][action] with the Q-learning rule.

            if terminated or truncated:
                break
            state = new_state
    return Qtable
```

[8]:
```
# Training parameters
n_training_episodes = 10000
learning_rate = 0.7

# Evaluation parameters
n_eval_episodes = 100

# Environment parameters
max_steps = 99
gamma = 0.95

# Exploration parameters
max_epsilon = 1.0
min_epsilon = 0.05
decay_rate = 0.0005
```

[9]:
```
start = time.time()
Qtable_frozenlake = train(n_training_episodes, min_epsilon, max_epsilon,␣
↪decay_rate,
                          env, max_steps, Qtable_frozenlake, learning_rate,␣
↪gamma)
```

```
print(f'Training took {time.time() - start:.1f}s')
print(Qtable_frozenlake.round(3))
```

To evaluate the agent we follow the **greedy** policy only (no exploration) and report the mean and standard deviation of the total reward per episode.

[10]:
```
def evaluate_agent(env: gym.Env, max_steps: int, n_eval_episodes: int, Q: np.
 ↪ndarray,
                   seed: list[int]) -> tuple[float, float]:
    episode_rewards = []
    for episode in range(n_eval_episodes):
        if seed:
            state, info = env.reset(seed=seed[episode])
        else:
            state, info = env.reset()
        total_rewards_ep = 0

        for step in range(max_steps):
            action = greedy_policy(Q, state)
            new_state, reward, terminated, truncated, info = env.step(action)
            total_rewards_ep += reward
            if terminated or truncated:
                break
            state = new_state
        episode_rewards.append(total_rewards_ep)
    return float(np.mean(episode_rewards)), float(np.std(episode_rewards))
```

[11]:
```
mean_reward, std_reward = evaluate_agent(env, max_steps, n_eval_episodes,␣
 ↪Qtable_frozenlake, [])
print(f'Mean reward = {mean_reward:.2f} +/- {std_reward:.2f}')

# The greedy policy as arrows on the grid (holes and goal have no meaningful␣
 ↪action).
arrows = np.array(['<', 'v', '>', '^'])[Qtable_frozenlake.argmax(axis=1)].
 ↪reshape(4, 4)
print(arrows)
```

You should see a mean reward of 1.0: the greedy policy reaches the goal every time.

Question: After training, many Q-values are exactly 0 - for example all four values of the hole states and of the goal state. Why?

Question: The non-terminal Q-values are powers of $\gamma = 0.95$, e.g. $0.95^5 \approx 0.774$ in the start state. Why?

1.4 Exercise 2: Taxi, and a Slippery Lake

The same code solves a bigger task. Taxi has a 5×5 grid with four marked locations R, G, Y, B. The taxi has to pick up a passenger at one of them and drop them off at another. There are $25 \cdot 5 \cdot 4 = 500$ states (taxi position, passenger location including "in the taxi", destination) and 6 actions (4 moves, pick-up, drop-off). The reward is -1 per step, $+20$ for a successful drop-off and -10 for an illegal pick-up or drop-off.

(Gymnasium 1.3 renamed `Taxi-v3` to `Taxi-v4`; the default behaviour is unchanged. The next cell uses whichever your installation has.)

[12]:
```
taxi_id = 'Taxi-v4' if 'Taxi-v4' in gym.registry else 'Taxi-v3'
taxi_env = gym.make(taxi_id, render_mode='ansi')
taxi_env.reset(seed=0)
taxi_env.action_space.seed(0)
print(taxi_env.render())
print('Observation space:', taxi_env.observation_space)
print('Action space:', taxi_env.action_space)
```

```
+---------+
|R: | : :G|
| : | : : |
| : : : : |
|▒|| : | : |
|Y| : |B: |
+---------+


Observation space: Discrete(500)
Action space: Discrete(6)
```

The evaluation seeds below are the source course's: they fix the 100 start configurations, so everyone's agent is evaluated on the same episodes.

[13]:
```
taxi_eval_seed = [16, 54, 165, 177, 191, 191, 120, 80, 149, 178, 48, 38, 6, 125,␣
↪174, 73, 50, 172,
                  100, 148, 146, 6, 25, 40, 68, 148, 49, 167, 9, 97, 164, 176,␣
↪61, 7, 54, 55, 161,
                  131, 184, 51, 170, 12, 120, 113, 95, 126, 51, 98, 36, 135, 54,␣
↪82, 45, 95, 89, 59,
                  95, 124, 9, 113, 58, 85, 51, 134, 121, 169, 105, 21, 30, 11,␣
↪50, 65, 12, 43, 82,
                  145, 152, 97, 106, 55, 31, 85, 38, 112, 102, 168, 123, 97, 21,␣
↪83, 158, 26, 80, 63,
                  5, 81, 32, 11, 28, 148]
```

[14]:
```
start = time.time()
Qtable_taxi = initialize_q_table(taxi_env.observation_space.n, taxi_env.
↪action_space.n)
```

```
Qtable_taxi = train(n_training_episodes=25000, min_epsilon=0.05, max_epsilon=1.0,
                    decay_rate=0.005, env=taxi_env, max_steps=99,␣
 ↪Qtable=Qtable_taxi,
                    learning_rate=0.7, gamma=0.95)
print(f'Training took {time.time() - start:.1f}s')
mean_reward, std_reward = evaluate_agent(taxi_env, 99, 100, Qtable_taxi,␣
 ↪taxi_eval_seed)
print(f'Mean reward = {mean_reward:.2f} +/- {std_reward:.2f}')
```

When this lab was tested, the mean reward was about 7.5 (the reward of an episode depends on how far the taxi has to drive, so the spread is not a sign of failure).

Back to FrozenLake, now **slippery**: an action moves the agent in the intended direction only with probability 1/3, and to either side otherwise. The MDP is now stochastic. Train the same agent with the same hyperparameters as in Exercise 1, and evaluate it on 1,000 episodes.

[15]:
```
slippery_env = gym.make('FrozenLake-v1', map_name='4x4', is_slippery=True)
slippery_env.reset(seed=0)
slippery_env.action_space.seed(0)
# TODO: Train a fresh Q-table on slippery_env with the Exercise 1␣
 ↪hyperparameters,
# then evaluate it on 1000 episodes (no seed list).
print(f'Mean reward = {mean_reward:.2f} +/- {std_reward:.2f}')
```

When this lab was tested, the mean reward dropped to about 0.3. Now try `learning_rate=0.1`, `gamma=0.99` and 20,000 training episodes.

[16]:
```
# TODO: Same as above with learning_rate=0.1, gamma=0.99 and 20000 episodes.
print(f'Mean reward = {mean_reward:.2f} +/- {std_reward:.2f}')
```

With these settings it rose to about 0.75 in testing.

Question: Why does a learning rate of 0.7 work well on the non-slippery lake but badly on the slippery one?

1.5 Exercise 3: REINFORCE on CartPole

Tabular Q-learning needs one table entry per state. In CartPole the state is continuous: cart position, cart velocity, pole angle and pole angular velocity. The agent pushes the cart left or right and gets a reward of +1 for every step the pole stays upright. An episode ends when the pole tilts more than 12, the cart leaves the track, or after 500 steps - so 500 is the maximum return.

Instead of approximating Q, we now parametrise the **policy** directly (Session 11, "Policy Gradient"): a small network $\pi_\theta(a|s)$ that outputs a probability for each action.

[17]:
```
cartpole_env = gym.make('CartPole-v1')
s_size = cartpole_env.observation_space.shape[0]
a_size = cartpole_env.action_space.n
print('State size:', s_size, ' Action size:', a_size)
print('Sample observation:', cartpole_env.observation_space.sample())
```

```
State size: 4  Action size: 2
Sample observation: [-3.9048336   0.62210083 -0.3032624  -0.8875389 ]
```

The policy network has two fully connected layers with a ReLU in between and a softmax at the end. `act` turns a state into an action: it must **sample** from $\pi_\theta(\cdot|s)$ - not take the most likely action - and also return the log probability $\log \pi_\theta(a|s)$ of the sampled action, which the policy gradient needs.

[18]:
```
class Policy(nn.Module):
    def __init__(self, s_size: int, a_size: int, h_size: int):
        super().__init__()
# TODO: Two linear layers: s_size -> h_size and h_size -> a_size.

    def forward(self, x: torch.Tensor) -> torch.Tensor:
# TODO: fc1, ReLU, fc2, then a softmax over the action dimension (dim=1).

    def act(self, state: np.ndarray) -> tuple[int, torch.Tensor]:
        state = torch.from_numpy(state).float().unsqueeze(0)
        probs = self.forward(state)
        m = Categorical(probs)
# TODO: Sample an action from m and return (action as int, its log probability).
```

[19]:
```
debug_policy = Policy(s_size, a_size, 16)
state, info = cartpole_env.reset(seed=0)
action, log_prob = debug_policy.act(state)
print(action, log_prob)
assert action in (0, 1) and log_prob.requires_grad
```

REINFORCE (Session 11, "Monte-Carlo Policy Gradient") plays a whole episode with the current policy, then updates θ in the direction of

$$\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)\, G_t,$$

where G_t is the return from step t onwards. It is an unbiased sample of $Q^{\pi_\theta}(s_t, a_t)$. Two implementation details:

- **Computing G_t for all t.** Use $G_t = r_{t+1} + \gamma G_{t+1}$ and go backwards through the episode, so each return reuses the next one.

- **Gradient ascent with a minimiser.** PyTorch optimisers minimise, so we minimise the loss $-\sum_t \log \pi_\theta(a_t|s_t)\, G_t$, whose gradient is the negative policy gradient.

As in the source course, the returns of an episode are standardised (mean 0, standard deviation 1) before the update.

[20]:
```
def reinforce(policy: Policy, optimizer: torch.optim.Optimizer, env: gym.Env,
              n_training_episodes: int, max_t: int, gamma: float, print_every:␣
↪int) -> list[float]:
    scores_deque = deque(maxlen=100)
    scores = []
    for i_episode in range(1, n_training_episodes + 1):
        saved_log_probs = []
        rewards = []
        state, info = env.reset()
        for t in range(max_t):
# TODO: Get an action and its log probability from the policy, take an env
# step, and store the log probability and the reward. Stop when the episode
# is terminated or truncated.
        scores_deque.append(sum(rewards))
        scores.append(sum(rewards))

        # The return G_t at every step, computed backwards: G_t = r_(t+1) +␣
↪gamma * G_(t+1).
        # appendleft keeps the deque in chronological order.
        returns = deque(maxlen=max_t)
        for t in range(len(rewards))[::-1]:
            disc_return_t = returns[0] if len(returns) > 0 else 0
# TODO: returns.appendleft(...) the return at step t.

        eps = np.finfo(np.float32).eps.item()
        returns = torch.tensor(returns)
        returns = (returns - returns.mean()) / (returns.std() + eps)

# TODO: The loss is the sum over t of -log_prob * return. Then do one optimizer
# step (zero_grad, backward, step).

        if i_episode % print_every == 0:
            print(f'Episode {i_episode}\tAverage score (last 100): {np.
↪mean(scores_deque):.1f}')
    return scores
```

[21]:
```
cartpole_hyperparameters = {
    'h_size': 16,
    'n_training_episodes': 500,
    'max_t': 1000,
```

```
    'gamma': 1.0,
    'lr': 1e-2,
}
```

[22]:
```
torch.manual_seed(0)
cartpole_env.reset(seed=0)
cartpole_policy = Policy(s_size, a_size, cartpole_hyperparameters['h_size'])
cartpole_optimizer = torch.optim.Adam(cartpole_policy.parameters(),␣
↪lr=cartpole_hyperparameters['lr'])

start = time.time()
scores_seed0 = reinforce(cartpole_policy, cartpole_optimizer, cartpole_env,
                         cartpole_hyperparameters['n_training_episodes'],
                         cartpole_hyperparameters['max_t'],␣
↪cartpole_hyperparameters['gamma'], 50)
print(f'Training took {time.time() - start:.0f}s')
```

Evaluate the trained policy on 10 fresh episodes. It still samples its actions; a well-trained policy is almost deterministic anyway.

[23]:
```
def evaluate_policy(env: gym.Env, max_steps: int, n_eval_episodes: int, policy:␣
↪Policy) -> tuple[float, float]:
    episode_rewards = []
    for episode in range(n_eval_episodes):
        state, info = env.reset()
        total_rewards_ep = 0
        for step in range(max_steps):
            action, _ = policy.act(state)
            state, reward, terminated, truncated, info = env.step(action)
            total_rewards_ep += reward
            if terminated or truncated:
                break
        episode_rewards.append(total_rewards_ep)
    return float(np.mean(episode_rewards)), float(np.std(episode_rewards))
```

[24]:
```
eval_env = gym.make('CartPole-v1')
eval_env.reset(seed=123)
mean_reward, std_reward = evaluate_policy(eval_env,␣
↪cartpole_hyperparameters['max_t'], 10, cartpole_policy)
print(f'Mean reward = {mean_reward:.1f} +/- {std_reward:.1f}')
```

Question: Standardising the returns subtracts their mean from every G_t. Which idea from Session 11 does this resemble, and why does subtracting a constant not change the expected gradient?

1.6 Exercise 4: One Run Is Not Enough

Session 11 listed "high variance" as a disadvantage of policy-based RL. Train the same agent with two more random seeds and plot all three learning curves (the score of each episode, smoothed over 20 episodes).

[25]:
```
all_scores = {0: scores_seed0}
for seed in (1, 2):
# TODO: Seed torch and the environment with `seed`, create a fresh Policy and␣
 ↪Adam
# optimizer, and train with the same hyperparameters. Store the scores in
# all_scores[seed].
```

[26]:
```
plt.figure(figsize=(8, 4))
for seed, scores in all_scores.items():
    smoothed = np.convolve(scores, np.ones(20) / 20, mode='valid')
    plt.plot(np.arange(len(smoothed)) + 20, smoothed, label=f'seed {seed}')
plt.axhline(500, color='gray', linestyle='--', linewidth=1)
plt.xlabel('episode')
plt.ylabel('score (moving average over 20 episodes)')
plt.legend()
plt.show()
for seed, scores in all_scores.items():
    print(f'seed {seed}: mean score of the last 100 episodes = {np.
 ↪mean(scores[-100:]):.1f}')
```

When this lab was tested, the three seeds ended between about 135 and 500 (mean of the last 100 episodes) - with identical code and hyperparameters. Curves also often reach 500 and then drop again.

Question: What does this mean for comparing two RL algorithms, or two settings of one algorithm, in a report?

1.7 Conclusion

Here's what you should take away from this lab:

- Tabular Q-learning (Exercises 1-2) is a few lines of code: an ϵ-greedy behaviour policy for exploration, a greedy target policy, and the update towards $R + \gamma \max_{a'} Q(S', a')$. On a deterministic lake its Q-values converge to the exact discounted returns; on a stochastic one the learning rate has to average over noisy targets.
- A lookup table does not scale to continuous states. REINFORCE (Exercise 3) instead learns a policy network directly from complete episodes, using $\log \pi_\theta$ times the return as its learning signal - the policy gradient theorem from Session 11 in code.
- Policy-gradient training is noisy (Exercise 4): the same code reaches very different results with different seeds. Comparisons need several seeds and a reported spread.